

EspBellAdmin

Project logic, component responsibilities, runtime flow, and ESP32-S3 hardware references.

Inspected scope: project files in `source/` and `doc/`, including web assets and chip-reference files.

Document status: generated from the repository as inspected on 2026-08-28.

Runtime: MicroPython on ESP32-S3 N16R8 with I2S audio output.

1. Purpose and Runtime Summary

EspBellAdmin is an embedded bell and audio appliance. It stores a target audio file on the ESP32 filesystem, plays it through a PCM5102A-compatible I2S DAC, exposes a browser control panel, and can shut down after a configurable period without HTTP activity.

Configured behavior: `boot_mode` is `music_first`, so boot audio completes before networking and the web server are started. The configured web endpoint is HTTPS on port 443.

2. Component Graph

```
MicroPython runtime
|
+--> source/main.py :: main()
    +--> load_config() -----> source/config.json
    +--> hal/boot_player.py -----> hal/audio_utils.py
        |
        +--> I2S TX -> PCM5102A DAC
    +--> setup_network() -----> STA Wi-Fi or AP fallback
        |
        +--> dns_thread() in AP mode
    +--> hal/power_manager.py -----> smart timeout -> GPIO4 shutdown
    +--> hal/indicator.py -----> NeoPixel status -> GPIO48 default
    +--> app/server.py :: init_server()
        +--> app/security.py -----> nonce + SHA-256 authentication
        +--> app/player.py -----> UI playback task
        +--> app/www/* -----> browser UI

Browser --> /api/info, /api/get-nonce, /api/verify-auth
        --> /api/config, /upload, /api/play, /api/stop, /api/logout
        --> /media/bell.wav --> audio_utils.py --> I2S --> DAC --> speaker
```

The graph shows implemented imports and calls. Chip diagnostics exist but their boot invocation is commented out.

util/initWiFi.py is a legacy standalone utility and is not imported by the active path.

3. Startup and Lifecycle

1. **Configuration:** [source/main.py](#) reads JSON after removing full-line `//` and `#` comments. Plain configured passwords are converted to ENC: values on first load.
2. **Filesystem:** required runtime directories are attempted before playback.
3. **Offline audio:** with `music_first`, [hal/boot_player.py](#) calls the synchronous wrapper around `play_audio_track(mode='boot')`.
4. **Network:** `setup_network()` tries station mode for the configured router. Failure activates an access point and starts a DNS redirect thread.
5. **Background tasks:** smart timeout and LED loop are scheduled on `uasyncio`.
6. **Web service:** [app/server.py](#) registers Microdot routes and starts HTTP or HTTPS using configured certificate and key paths.

4. Audio Pipeline

Stage	Implementation	Behavior
Selection	hal/audio_utils.py	Uses <code>media_dir</code> and <code>target_filename</code> ; config supplies I2S ID and GPIOs.
WAV	<code>parse_wav_header()</code> , <code>stream_wav()</code>	Parses RIFF chunks, supports mono-to-stereo duplication, gain scaling, fade-out, duration limits, position tracking, and frame-aligned resume.
MP3	<code>stream_mp3()</code> plus custom <code>mp3dec</code>	Decodes chunks to PCM and writes stereo I2S. The custom decoder must exist in firmware.
Output	<code>init_hardware_i2s()</code>	Creates MicroPython I2S TX with BCK=15, WS=16, SD=17 by default, 16-bit stereo output, and a DMA buffer.
UI control	app/player.py	Starts asynchronous UI playback in mode <code>ui</code> ; stop sets a flag and deinitializes I2S.

Observed limitation: MP3 resume uses an approximate 16,000 file-bytes-per-second model and can seek into a compressed frame. Uploaded browser files are converted to WAV, so the normal UI upload path primarily exercises WAV playback.

5. Web UI and API

Browser application

- [index.html](#) loads [app.js](#).

- `app.js` loads login, main, upload, and config views dynamically.
- Selected MP3/WAV files are decoded in the browser and encoded as PCM WAV before upload.
- [style.css](#) supplies responsive presentation.

Routes

- Public: `/`, `/www/*`, `/api/info`, `/api/get-nonce`, `/api/logout`.
- Protected: `/api/config`, `/api/play`, `/api/stop`, `/upload`.
- Catch-all redirects to the configured AP IP.
- Intended `/api/logs` registration is after return `app` and is unreachable.

6. Authentication and Configuration

[SecurityManager](#) creates an 8-byte random nonce, stores it with a TTL, and accepts `SHA256(password + nonce)`. A nonce is deleted before hash comparison, making each token single-use. Password encryption uses AES-128-CBC when `ucryptolib` is available, with a repeating-key XOR fallback.

[config.json](#) controls boot mode, timeouts, repeat and fade behavior, resume position, Wi-Fi/AP values, file limits, HTTPS paths, media paths, and I2S pins. The server updates selected values in place with regular expressions to retain comments.

Security observations: the checked-in configuration contains default plaintext credentials; the browser retains the entered password for the session; authenticated configuration responses include decrypted passwords; and the XOR fallback is not cryptographically secure. Secret contents are intentionally excluded from this document.

7. Hardware Map and Reference Files

Function	Software assignment	Reference / interpretation
I2S BCK	GPIO15	Maps to PCM5102A BCK. See PCM5102A_DAC_Pins.jpg .
I2S WS/LRCK	GPIO16	Maps to PCM5102A LRCK/WS.
I2S serial data	GPIO17	Maps to PCM5102A DIN.
Power / relay shutdown	GPIO4	Used by power_manager.py to drive LOW. Relay startup latch code is commented out. See Relay ModulePins.jpeg .
Onboard NeoPixel	Default GPIO48 in software	ESP32_Pins.jpg labels the onboard RGB LED as GPIO47. This is an unresolved board/software mismatch.

Additional board and DAC references: [ESP32_Pins.avif](#), [Esp32_Board.jpg](#), [Esp32OnBoard.jpg](#), [ESp32_Chip.jpg](#), [pcm5102a_exact_board_pinout_good.pdf](#), [pcm5102a_esp32s3_pinout.pdf](#),

[pcm5102a_expansion_board_pinout.pdf](#), [pcm5102a_visual_Good.pdf](#), [pcm5102a_visual_guide.pdf](#), [pi.jpg](#), and [RelayModuleDesc.avif](#).

8. Supporting Components and File Inventory

Area	Files	Status
Entrypoint and logging	main.py , boot.py , logger.py	Active entrypoint and logging; boot.py is empty; log rotation targets /boot . log.
Application	server.py , player.py , security.py	Active web, playback, and auth layers.
HAL	audio_utils.py , boot_player.py , power_manager.py , indicator.py , chip_monitor.py	Active audio/power/indicator code; chip diagnostics exist but boot call is disabled.
Libraries and resources	lib/microdot.py , lib/logging.py , resources/cert.crt, resources/cert.key	Vendored framework and MicroPython logging; TLS material is referenced without exposing contents.
Legacy utility	util/initWiFi.py	Standalone experimental AP/raw upload server; import-time infinite loop; not active.

9. Risks and Follow-up Checks

- Confirm whether the board onboard LED is GPIO47 or GPIO48, then set one authoritative value.
- Move or register the logs route before returning the Microdot app if logs are intended to be exposed.
- Replace regex/string configuration persistence with structured JSON serialization that escapes strings and preserves boolean types.
- Revisit MP3 resume semantics and ensure stop/deinitialization cannot race the active playback task.
- Review TLS certificate deployment, default credentials, and whether decrypted passwords should ever be returned to the browser.
- On real hardware, validate GPIO4 relay polarity and the complete power-latch circuit; startup latch implementation is disabled.

Validation: architecture statements were cross-checked against the active source path and hardware reference inventory. This document describes the implementation as found; it does not imply hardware-only behavior has been bench-tested.